Міністерство освіти і науки України
Київський національний університет імені Тараса Шевченка

ЗВІТ

про виконання лабораторної роботи №1

з дисципліни

«Організація баз даних»

Тема: «Контекстна, логічна та фізична моделі бази даних системи моніторингу мережевого обладнання та ДБЖ NetGuard Monitor»

Перевірив:

_____

Виконав:
студент 3 курсу
групи ІР-31
Микитенко Ілля Андрійович

Київ 2026

1. Мета та завдання роботи

Мета роботи — набути практичних навичок проєктування реляційної бази даних (БД) на трьох рівнях абстракції — контекстному (концептуальному), логічному та фізичному — на прикладі БД системи моніторингу мережевого обладнання та джерел безперебійного живлення (ДБЖ) інтернет-провайдера NetGuard Monitor, а також сформулювати функціональні вимоги до програмної реалізації спроєктованої БД, яку буде виконано в лабораторній роботі №2.

Завдання роботи:

- проаналізувати предметну область і визначити інформаційні потреби користувачів та сервісів системи;
- побудувати контекстну модель БД: контекстну діаграму потоків даних і ER-модель у нотації Чена;
- побудувати логічну модель: перетворити ER-модель на реляційну схему, нормалізувати відношення щонайменше до третьої нормальної форми (ЗНФ), визначити ключі та обмеження цілісності;
- побудувати фізичну модель для СУБД PostgreSQL: обрати типи даних, індекси й спосіб секціонування, оцінити обсяг даних, розробити DDL-скрипт і перевірити його на СУБД;
- описати таблиці БД із зазначенням типів даних та обмежень;
- сформулювати висновки та функціональні вимоги до програмної реалізації БД.

Вихідні дані — специфікація вимог (SRS), архітектура IoT-рішення та моделі UML/IDEF1X, розроблені в лабораторних роботах №1–4 з дисципліни «Технології створення програмних продуктів» (далі — ЛР1–ЛР4 ТСПП): функціональні вимоги FR-01…FR-12, системні вимоги SR-04.1…SR-10.1, нефункціональні вимоги NFR-01…NFR-06 і попередня структура БД із 8 таблиць.

2. Аналіз предметної області

2.1 Опис предметної області

NetGuard Monitor — внутрішня система малого інтернет-провайдера, що має один центр обробки даних (ЦОД) і кілька районних вузлів доступу. Сервіс опитування (Polling Service) щонайменше раз на 60 с зчитує за протоколом SNMP показники активного мережевого обладнання (комутатори, маршрутизатори, ONU) і ДБЖ (заряд і напруга АКБ, вхідна та вихідна напруга, навантаження, температура). Сервіс аналізу (Alerting Service) порівнює показники з пороговими значеннями, які адміністратор задає для типу обладнання, і в разі порушення реєструє інцидент. Інцидент проходить життєвий цикл «новий → (ескальований) → прийнятий → усунений → закритий»; про нього сервіс сповіщень (Notification Service) інформує інженерів через веб-панель, e-mail і Telegram. Зміни стану, дії користувачів і зміни конфігурації фіксуються в журналі подій, на основі якого та історії показників формуються звіти.

БД є спільним сховищем для всіх мікросервісів системи (архітектура, ЛР2 ТСПП) і повинна забезпечувати: цілісність даних про обладнання та інциденти; швидкий запис потоку телеметрії від парку до 500 одиниць обладнання (NFR-01); швидке читання поточного стану та історії показників; зберігання історії телеметрії протягом року на сервері з диском 100 ГБ (NFR-02).

2.2 Інформаційні потреби

Інформаційні потреби виведено з функціональних і системних вимог SRS: для кожної вимоги визначено дані, які БД повинна зберігати (табл. 1).

Таблиця 1 — Інформаційні потреби системи

Вимога	Інформаційна потреба	Дані, що зберігаються в БД
FR-01, FR-11	Автентифікація, ролі, блокування користувачів	Користувачі, ролі, хеш пароля, ознака активності
FR-02	Облік мережевого обладнання	Обладнання, типи, вузли (локації), параметри SNMP
FR-03	Облік ДБЖ	Паспортні характеристики ДБЖ: потужність, ємність і кількість АКБ
FR-04, SR-04.1, SR-04.2	Збір телеметрії, облік недоступності	Показники з часовою міткою, лічильник невдалих опитувань
FR-05	Поточний стан обладнання	Статус пристрою, час останньої відповіді, останні значення метрик
FR-06	Порогові значення	Межі норми для пари «тип обладнання — метрика», рівень критичності
FR-07, SR-07.1, SR-07.2	Сповіщення, антифлуд	Сповіщення за каналами, стан доставки, час відправлення
FR-08	Обробка інцидентів	Інциденти, статус, моменти переходів, відповідальний, коментар
FR-09	Журнал подій	Події з типом, часом, користувачем, обладнанням, деталями
FR-10, SR-10.1	Звітність	Агрегати на основі інцидентів, показників і журналу
FR-12	Графіки показників	Історія значень метрики пристрою за період

2.3 Уточнення попередньої структури даних

Попередня структура БД (додаток А ЛР1 ТСПП, модель IDEF1X ЛР4 ТСПП) містила 8 таблиць. Під час проєктування на рівні дисципліни «Організація баз даних» її уточнено (табл. 2); у результаті БД складається з 12 таблиць.

Таблиця 2 — Зміни відносно попередньої структури

Елемент попередньої структури	Зміна	Обґрунтування
Текстове поле `metric_name` у `threshold` і `metric_reading`	Довідник `metric` (код, назва, одиниця виміру)	Усунення дублювання та аномалій оновлення (3НФ); SMALLINT замість рядка зменшує найбільшу таблицю
Атрибути ДБЖ (FR-03) не мали місця зберігання	Підтип `ups_details` (1:1 з `equipment`)	Характеристики ДБЖ не притаманні комутаторам; без підтипу більшість рядків мала б порожні поля
`incident.threshold_id` обов’язковий	Допускає NULL; додано тип інциденту і моменти переходів статусів	Інцидент недоступності (SR-04.2) не пов’язаний з порогом; діаграма станів потребує фіксації часу переходів
Сповіщення не зберігалися	Таблиця `notification`	FR-07; антифлуд (SR-07.2) і ескалація потребують історії відправлень
Журнал подій не мав таблиці	Таблиця `event_log`	FR-09: фільтрація подій за датою, вузлом, типом

Елемент попередньої структури	Зміна	Обґрунтування
Сурогатний ключ `metric_reading.id`	Природний ключ (`equipment_id`, `metric_id`, `read_at`)	Економія 8 байт і одного індексу на рядок; ключ збігається з основним запитом — історією показника

3. Контекстна модель бази даних

Контекстна (концептуальна, інфологічна) модель описує БД на найвищому рівні абстракції, незалежно від СУБД: які об’єкти предметної області відображаються в БД, хто її наповнює та використовує і як об’єкти пов’язані між собою. Модель складається з контекстної діаграми, що визначає межі БД та потоки даних, і ER-діаграми в нотації Чена.

3.1 Контекстна діаграма

На контекстній діаграмі (рис. 1) БД зображено єдиним сховищем у межах контуру «Межа бази даних». Безпосередньо з БД працюють лише п’ять сервісів системи. Люди (інженер, адміністратор) і зовнішні системи (SNMP-агенти, канали доставки повідомлень) взаємодіють з даними тільки через сервіси. Тому доступ до даних можна контролювати на рівні СУБД через окремі облікові записи сервісів.

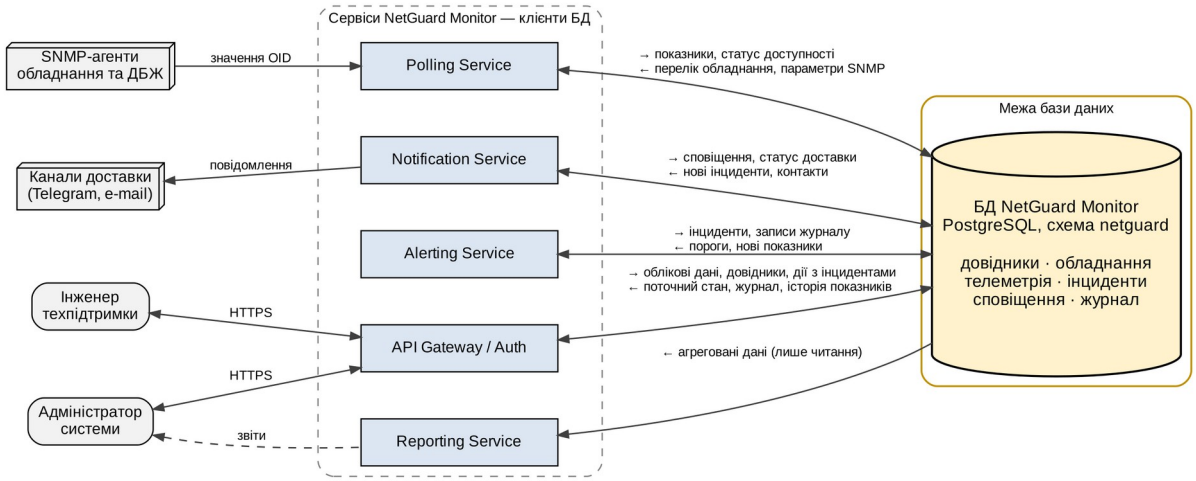


Рисунок 1 — Контекстна діаграма бази даних NetGuard Monitor

Позначення «→» на потоці — дані, що записуються в БД; «←» — дані, що читаються з БД. Перелік потоків і таблиць, які вони зачіпають, наведено в табл. 3.

Таблиця 3 — Потоки даних між клієнтами та БД

Клієнт БД	Записує в БД	Читає з БД	Основні таблиці
Polling Service	Показники телеметрії, статус пристрою, лічильник невдалих опитувань	Перелік активного обладнання, параметри SNMP	`equipment`, `metric_reading`
Alerting Service	Інциденти, записи журналу	Активні пороги, нові показники	`threshold`, `metric_reading`, `incident`, `event_log`
Notification Service	Сповіщення, стан доставки	Нові інциденти, контакти та ролі користувачів	`incident`, `notification`, `users`

Клієнт БД	Записує в БД	Читає з БД	Основні таблиці
API Gateway / Auth	Користувачі, довідники, обладнання, пороги, дії з інцидентами	Поточний стан, інциденти, журнал, історія показників	Усі таблиці
Reporting Service	— (лише читання)	Агрегати інцидентів, показників і журналу	`incident`, `metric_reading`, `event_log`

3.2 Сутності предметної області

За результатами аналізу інформаційних потреб виділено 12 сутностей (табл. 4). Для кожної визначено природний ідентифікатор — атрибут або сукупність атрибутів, що однозначно визначає екземпляр сутності в предметній області.

Таблиця 4 — Сутності предметної області

Сутність	Опис	Природний ідентифікатор	Таблиця
РОЛЬ	Набір повноважень користувача («Сапорт», «Адміністратор»)	Код ролі	`roles`
КОРИСТУВАЧ	Інженер або адміністратор, що працює з системою	Логін	`users`
ЛОКАЦІЯ	Вузол провайдера: ЦОД або районний вузол доступу	Назва	`locations`
ТИП ОБЛАДНАННЯ	Модель/клас пристроїв зі спільним профілем SNMP і порогами	Назва	`equipment_type`
ОБЛАДНАННЯ	Конкретний пристрій, що опитується за SNMP	IP-адреса; ім’я	`equipment`
ДБЖ	Уточнення (підтип) обладнання з паспортними характеристиками	Ідентифікатор обладнання	`ups_details`
МЕТРИКА	Вимірювана величина з одиницею виміру	Код	`metric`
ПОРІГ	Межі норми метрики для типу обладнання й рівня критичності	Тип + метрика + рівень	`threshold`
ПОКАЗНИК	Значення метрики пристрою в певний момент часу	Обладнання + метрика + час	`metric_reading`
ІНЦИДЕНТ	Зафіксоване відхилення показника або недоступність пристрою	Номер	`incident`
СПОВІЩЕННЯ	Повідомлення користувачу про інцидент певним каналом	Номер	`notification`
ПОДІЯ ЖУРНАЛУ	Запис про зміну стану, дію користувача, зміну конфігурації	Номер	`event_log`

3.3 ER-діаграма в нотації Чена

ER-діаграму наведено на рис. 2. Прямокутник позначає сутність, подвійний прямокутник — слабку сутність, що ідентифікується через власника; ромб — зв’язок, подвійний ромб — ідентифікуючий зв’язок; біля ліній указано кардинальність (1, N, 0..1). Атрибути на діаграмі не показано, щоб вона лишалася читабельною: їх наведено в логічній моделі (розділ 4).

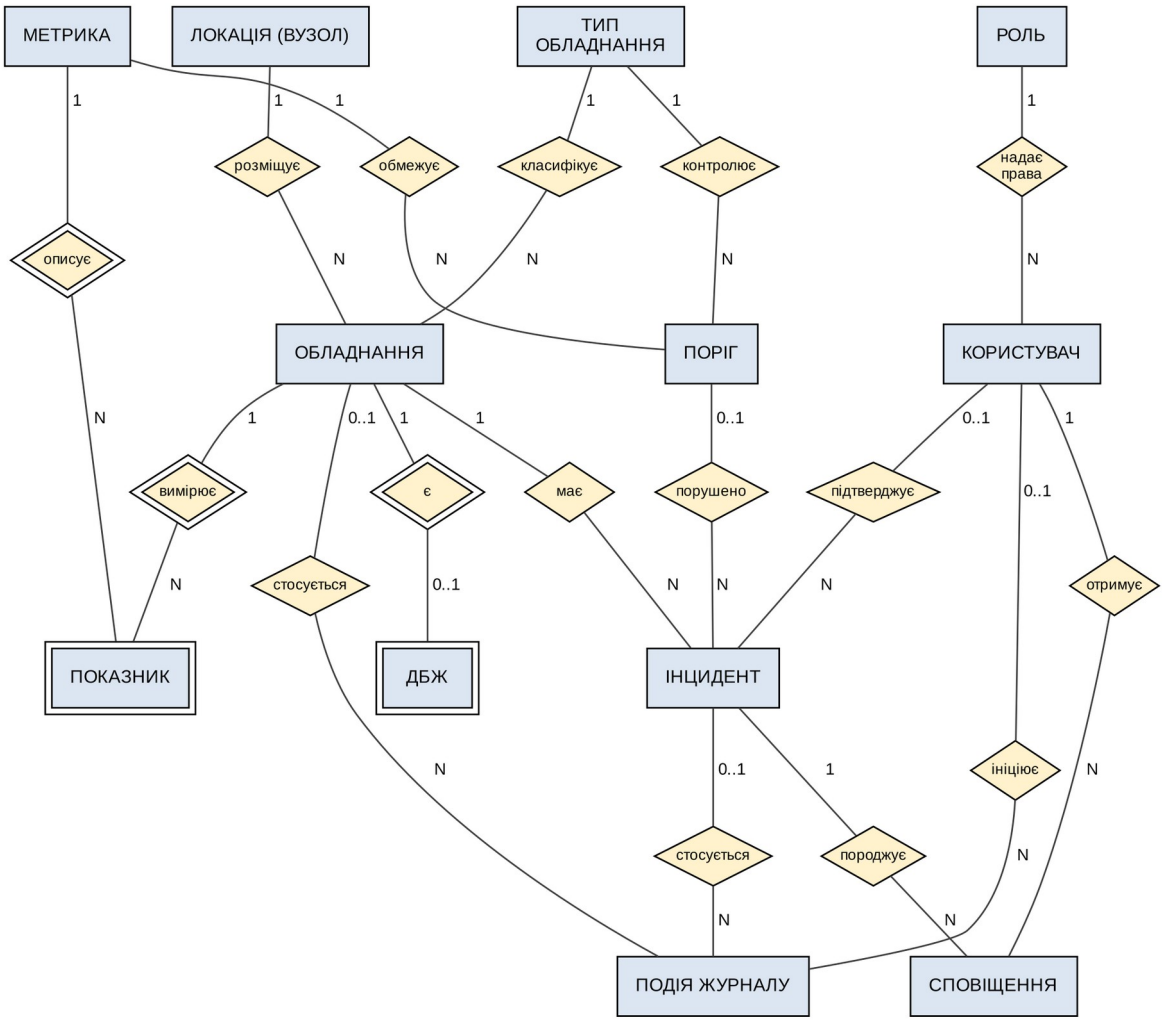


Рисунок 2 — Концептуальна ER-модель (нотація Чена)

Таблиця 5 — Зв’язки між сутностями

Зв’язок	Сутності	Кардинальність	Участь / примітка
надає права	РОЛЬ — КОРИСТУВАЧ	1 : N	Кожен користувач обов’язково має роль
розміщує	ЛОКАЦІЯ — ОБЛАДНАННЯ	1 : N	Обов’язкова для обладнання
класифікує	ТИП ОБЛАДНАННЯ — ОБЛАДНАННЯ	1 : N	Обов’язкова для обладнання
є (уточнення)	ОБЛАДНАННЯ — ДБЖ	1 : 0..1	Ідентифікуючий; ДБЖ не існує без обладнання
контролює	ТИП ОБЛАДНАННЯ — ПОРІГ	1 : N	Обов’язкова для порогу
обмежує	МЕТРИКА — ПОРІГ	1 : N	Обов’язкова для порогу
вимірює	ОБЛАДНАННЯ — ПОКАЗНИК	1 : N	Ідентифікуючий
описує	МЕТРИКА — ПОКАЗНИК	1 : N	Ідентифікуючий
має	ОБЛАДНАННЯ — ІНЦИДЕНТ	1 : N	Обов’язкова для інциденту
порушено	ПОРІГ — ІНЦИДЕНТ	0..1 : N	Лише для інцидентів типу «поріг»
підтверджує	КОРИСТУВАЧ — ІНЦИДЕНТ	0..1 : N	З моменту прийняття інциденту
породжує	ІНЦИДЕНТ — СПОВІЩЕННЯ	1 : N	Обов’язкова для сповіщення

Зв’язок	Сутності	Кардинальність	Участь / примітка
отримує	КОРИСТУВАЧ — СПОВІЩЕННЯ	1 : N	Обов’язкова для сповіщення
ініціює	КОРИСТУВАЧ — ПОДІЯ ЖУРНАЛУ	0..1 : N	Системні події не мають ініціатора
стосується	ОБЛАДНАННЯ — ПОДІЯ ЖУРНАЛУ	0..1 : N	Необов’язкова
стосується	ІНЦИДЕНТ — ПОДІЯ ЖУРНАЛУ	0..1 : N	Необов’язкова

Зв’язків «багато-до-багатьох» без атрибутів у моделі немає. ПОРІГ по суті є асоціативною сутністю між ТИПОМ ОБЛАДНАННЯ та МЕТРИКОЮ (зв’язок M:N «тип контролюється за метрикою» з атрибутами меж і рівня критичності), а ПОКАЗНИК — асоціативною сутністю між ОБЛАДНАННЯМ і МЕТРИКОЮ з атрибутами часу та значення. Тому вже на концептуальному рівні їх подано як сутності, що спрощує перехід до реляційної моделі.

4. Логічна модель бази даних

Логічна модель описує структуру даних у термінах реляційної моделі — відношень, атрибутів, ключів і обмежень — без прив’язки до конкретної СУБД.

4.1 Правила переходу від ER-моделі до реляційної схеми

1) Кожна сутність перетворюється на відношення (таблицю). Для сутностей із довгими або змінюваними природними ідентифікаторами (логін, IP-адреса, назва) уведено сурогатний первинний ключ `id`, а природний ідентифікатор збережено як альтернативний ключ (UNIQUE).
2) Зв’язок 1:N реалізується зовнішнім ключем (FK) на боці «N». Необов’язкова участь (0..1) — зовнішнім ключем, що допускає NULL.
3) Зв’язок 1:1 «ОБЛАДНАННЯ — ДБЖ» (уточнення) реалізується спільним первинним ключем: `ups_details.equipment_id` одночасно є PK і FK.
4) Слабка сутність ПОКАЗНИК отримує складений первинний ключ із ключів власників і часової мітки: (`equipment_id`, `metric_id`, `read_at`).
5) Асоціативна сутність ПОРІГ стає окремим відношенням з двома FK і альтернативним ключем (`equipment_type_id`, `metric_id`, `severity`).
6) Багатозначні атрибути (кілька каналів сповіщення для одного інциденту) не зберігаються списком в одному полі, а утворюють окремі рядки таблиці `notification` (вимога 1НФ).

4.2 Логічна ER-діаграма

Логічну модель у нотації «воронячої лапки» (crow’s foot) наведено на рис. 3. Позначення: «||» — рівно один, «o|» — нуль або один, «o<» — нуль або багато; суцільна лінія — обов’язковий зовнішній ключ (NOT NULL), пунктирна — необов’язковий. Атрибути первинного ключа підкреслено й відокремлено лінією, зовнішні ключі виділено курсивом, «(O)» позначає необов’язковий атрибут (допускає NULL).

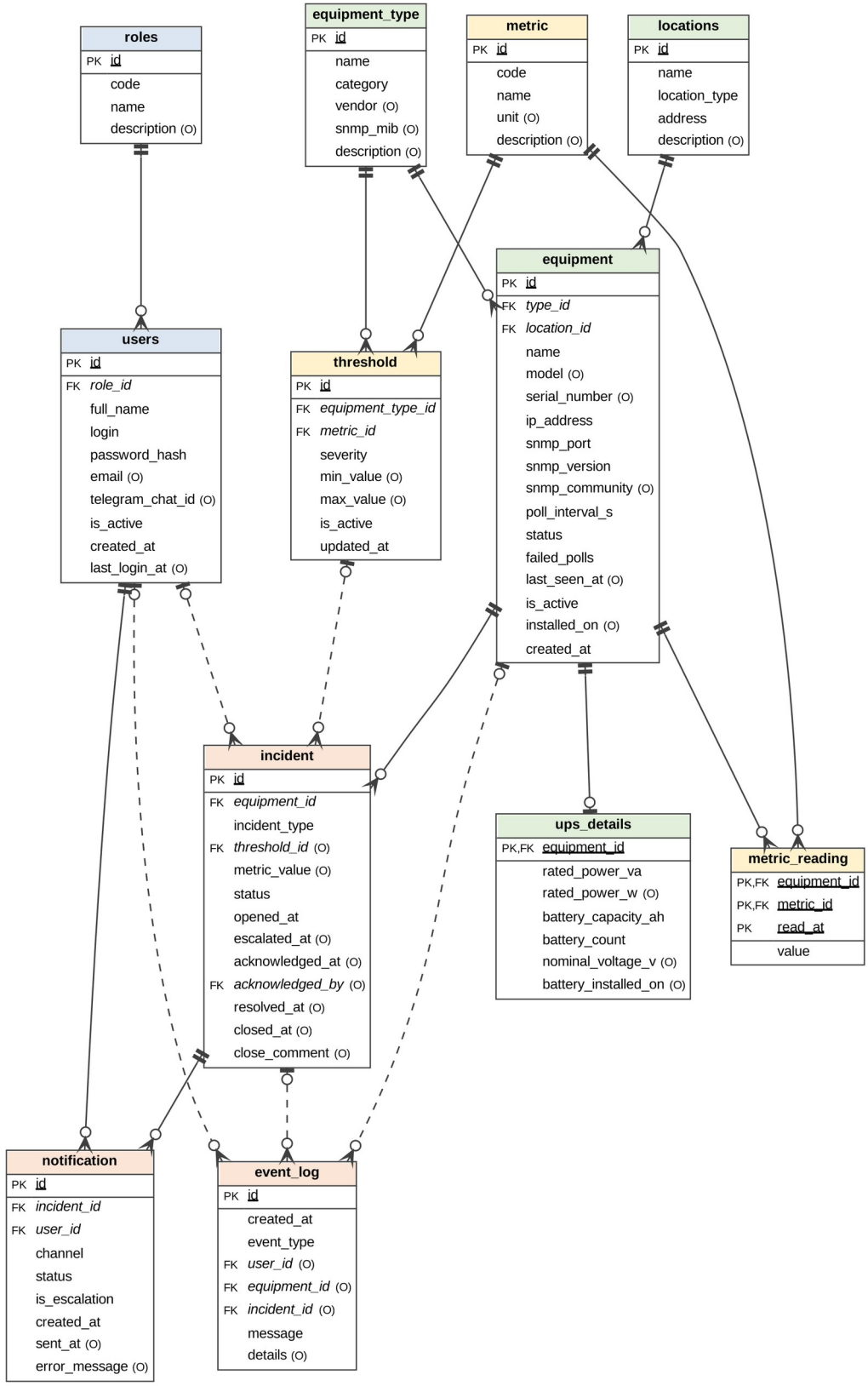


Рисунок 3 — Логічна модель бази даних (нотація crow’s foot)

4.3 Реляційна схема

Реляційна схема БД (первинні ключі підкреслено, зовнішні — курсивом):

`roles` (id, code, name, description)

users (id, *role_id*, full_name, login, password_hash, email, telegram_chat_id, is_active, created_at, last_login_at)

locations (id, name, location_type, address, description)

equipment_type (id, name, category, vendor, snmp_mib, description)

metric (id, code, name, unit, description)

equipment (id, *type_id*, *location_id*, name, model, serial_number, ip_address, snmp_port, snmp_version, snmp_community, poll_interval_s, status, failed_polls, last_seen_at, is_active, installed_on, created_at)

ups_details (*equipment_id*, rated_power_va, rated_power_w, battery_capacity_ah, battery_count, nominal_voltage_v, battery_installed_on)

threshold (id, *equipment_type_id*, *metric_id*, severity, min_value, max_value, is_active, updated_at)

metric_reading (*equipment_id*, *metric_id*, read_at, value)

incident (id, *equipment_id*, incident_type, *threshold_id*, metric_value, status, opened_at, escalated_at, acknowledged_at, *acknowledged_by*, resolved_at, closed_at, close_comment)

notification (id, *incident_id*, *user_id*, channel, status, is_escalation, created_at, sent_at, error_message)

event_log (id, created_at, event_type, *user_id*, *equipment_id*, *incident_id*, message, details)

4.4 Нормалізація

Перша нормальна форма (1НФ). Усі атрибути атомарні, повторюваних груп немає: кожне значення показника — окремий рядок `metric_reading`, кожне сповіщення за кожним каналом — окремий рядок `notification`. Поле `event_log.details` типу JSON розглядається як атомарне значення: це неструктурований додаток до запису журналу, який не бере участі в зв'язках і не дублює інших атрибутів.

Друга нормальна форма (2НФ). Часткові залежності можливі лише у відношеннях зі складеним ключем. У `metric_reading` єдиний неключовий атрибут `value` залежить від усього ключа (`equipment_id`, `metric_id`, `read_at`) — це значення конкретної метрики конкретного пристрою в конкретний момент. У `threshold` атрибути `min_value`, `max_value`, `is_active` повністю залежать від альтернативного ключа (`equipment_type_id`, `metric_id`, `severity`).

Третя нормальна форма (3НФ). Транзитивних залежностей неключових атрибутів немає. Зокрема: назва й одиниця виміру залежать від метрики, а не від показника, тому винесені в `metric`; адреса вузла залежить від локації, тому в `equipment` зберігається лише `location_id`; рівень критичності інциденту визначається порогом (`incident.threshold_id → threshold.severity`) і в `incident` не дублюється; локація події журналу визначається через `equipment_id → equipment.location_id` і в `event_log` не зберігається.

Нормальна форма Бойса — Кодда (НФБК). Кожен детермінант функціональної залежності є потенційним ключем. Наприклад, у `users` детермінантами є `id`, `login`, `email` і `telegram_chat_id` — усі вони потенційні ключі (UNIQUE); в `equipment` — `id`, `name`, `ip_address`, `serial_number`. Залежностей від неключових атрибутів не виявлено, отже схема перебуває в НФБК.

Контрольована денормалізація. Поле `incident.metric_value` — знімок значення в момент відкриття інциденту. Його зберігають навмисно: показники видаляються політикою зберігання (п. 5.6), а значення має лишатися, доки існує інцидент. Поля `equipment.status`, `failed_polls` і `last_seen_at` можна було б обчислювати з

телеметрії, але їх зберігають для швидкого відображення стану (FR-05) і для реалізації правила SR-04.2; актуальність цих полів підтримує сервіс опитування.

4.5 Обмеження цілісності

Цілісність сутностей забезпечується первинним ключем кожної таблиці. **Доменна цілісність** — типами даних, обмеженнями NOT NULL і CHECK та перелічуваними типами (п. 5.3–5.4). **Референційна цілісність** — зовнішніми ключами з правилами видалення, обраними так, щоб історія аварій і дій користувачів не втрачалася (табл. 6).

Таблиця 6 — Зовнішні ключі та правила референційної цілісності

Зовнішній ключ	Зв’язок	ON DELETE	Обґрунтування
`fk_users_role`	users → roles	RESTRICT	Не можна видалити роль, призначену користувачам (ON UPDATE CASCADE)
`fk_equipment_type`	equipment → equipment_type	RESTRICT	Тип, що використовується, не видаляється
`fk_equipment_location`	equipment → locations	RESTRICT	Вузол з обладнанням не видаляється
`fk_ups_equipment`	ups_details → equipment	CASCADE	Паспорт ДБЖ не існує без обладнання
`fk_threshold_type`	threshold → equipment_type	CASCADE	Пороги типу видаляються разом із типом (якщо на них не посилаються інциденти)
`fk_threshold_metric`	threshold → metric	RESTRICT	Метрика з порогами не видаляється
`fk_reading_equipment`	metric_reading → equipment	CASCADE	Телеметрія видаляється разом з обладнанням без інцидентів
`fk_reading_metric`	metric_reading → metric	RESTRICT	Метрика з історією не видаляється
`fk_incident_equipment`	incident → equipment	RESTRICT	Збереження історії аварій; обладнання виводиться з експлуатації
`fk_incident_threshold`	incident → threshold	RESTRICT	Поріг з історією деактивується (`is_active`)
`fk_incident_ack_user`	incident → users	RESTRICT	Відповідальний за інцидент не може зникнути; користувач блокується
`fk_notification_incident`	notification → incident	CASCADE	Сповіщення не мають сенсу без інциденту
`fk_notification_user`	notification → users	RESTRICT	Історія сповіщень користувача зберігається
`fk_event_log_user`, `fk_event_log_equipment`, `fk_event_log_incident`	event_log → users, equipment, incident	SET NULL	Запис журналу зберігається, посилання обнуляється

Бізнес-правила предметної області та механізми, якими їх забезпечено в БД, наведено в табл. 7. Правила, що потребують процедурної логіки (лічильники, таймери), буде реалізовано в лабораторній роботі №2 збереженими процедурами та тригерами, а в БД для них передбачено поля й індекси.

Таблиця 7 — Бізнес-правила та механізми їх забезпечення

Код	Бізнес-правило	Механізм у БД
БП-01	Кожен користувач має рівно одну роль	`role_id` NOT NULL, `fk_users_role`

Код	Бізнес-правило	Механізм у БД
БП-02	Логін унікальний: 3–50 малих латинських літер, цифр, «_», «.»	`uq_users_login`, `ck_users_login`
БП-03	Обладнання належить рівно одному вузлу та одному типу	NOT NULL, `fk_equipment_location`, `fk_equipment_type`
БП-04	IP-адреса обладнання унікальна і є адресою вузла, а не мережі	`uq_equipment_ip`, `ck_equipment_ip_host`
БП-05	Інтервал опитування — не більше 60 с (FR-04)	`ck_equipment_interval`
БП-06	Не більше одного паспорта ДБЖ на пристрій; активна потужність не перевищує повну	`pk_ups_details` (= FK), `ck_ups_power_w`; категорія `ups` — тригер (ЛР2)
БП-07	Поріг задає хоча б одну межу, min < max; один поріг на (тип, метрика, рівень)	`ck_threshold_bounds`, `ck_threshold_range`, `uq_threshold`
БП-08	Одне значення на (пристрій, метрика, момент); значення скінченне	`pk_metric_reading`, `ck_reading_value_finite`
БП-09	Інцидент типу «поріг» посилається на поріг і містить значення; «офлайн» — без порогу	`ck_incident_threshold`, `ck_incident_value`
БП-10	Не більше одного відкритого інциденту на (пристрій, поріг) або (пристрій, офлайн)	Частковий унікальний індекс `uq_incident_open`
БП-11	Статус інциденту узгоджений з моментами переходів; прийнятий має відповідального, закритий — коментар	`ck_incident_ack`, `ck_incident_resolved`, `ck_incident_closed`, `ck_incident_escalated`, `ck_incident_time_order`
БП-12	Надіслане сповіщення має час відправлення, невдале — текст помилки	`ck_notification_sent`, `ck_notification_failed`, `ck_notification_time`
БП-13	Історія не втрачається: обладнання та користувачі з історією лише деактивуються	ON DELETE RESTRICT, поля `is_active`
БП-14	Три поспіль невдалі опитування → «офлайн» та інцидент (SR-04.2); ескалація через 5 хв; антифлуд 10 хв (SR-07.2)	Поля `failed_polls`, `escalated_at`; індекси `ix_incident_new`, `ix_notification_antiflood`; процедури (ЛР2)

5. Фізична модель бази даних

5.1 Вибір СУБД і загальні рішення

Фізичну модель розроблено для СУБД **PostgreSQL 15+**, обраної в нефункціональних вимогах (NFR-03). На користь PostgreSQL свідчать повна підтримка транзакцій ACID і декларативних обмежень, спеціалізовані типи (`INET` для IP-адрес, `JSONB`, `TIMESTAMPTZ`, перелічувані типи), декларативне секціонування таблиць, часткові індекси та індекси BRIN для часових рядів, а також конструкція `NULLS NOT DISTINCT` (з версії 15), використана в індексі `uq_incident_open`. СУБД безкоштовна і працює on-premise у Docker-контейнері на сервері ЦОД (ЛР2 ТСПП).

Об’єкти БД розміщено в окремій схемі `netguard`. Правила іменування: `snake_case` латиницею; назви таблиць збережено з попередніх робіт (`users` — у множині, оскільки `user` — зарезервоване слово SQL); префікси обмежень та індексів — `pk_`, `fk_`, `uq_`, `ck_`, `ix_`. DDL-скрипт фізичної моделі (`sql/schema.sql`, додаток А) перевірено на PostgreSQL 16: його застосовано до чистої БД, після чого 25 тестових некоректних вставок і змін було відхилено відповідними обмеженнями, а повний життєвий цикл інциденту (new → acknowledged → resolved → closed) пройшов успішно.

5.2 Фізичні діаграми

Фізичну модель через великий обсяг подано двома діаграмами за предметними підобластями: «Обладнання та телеметрія» (рис. 4) та «Користувачі, інциденти, сповіщення» (рис. 5); таблиці з іншої підобласті показано пунктирними заглушками. Позначення: NN — NOT NULL, UQ — UNIQUE, ID — GENERATED ALWAYS AS IDENTITY, «partitioned» — секціонована таблиця.

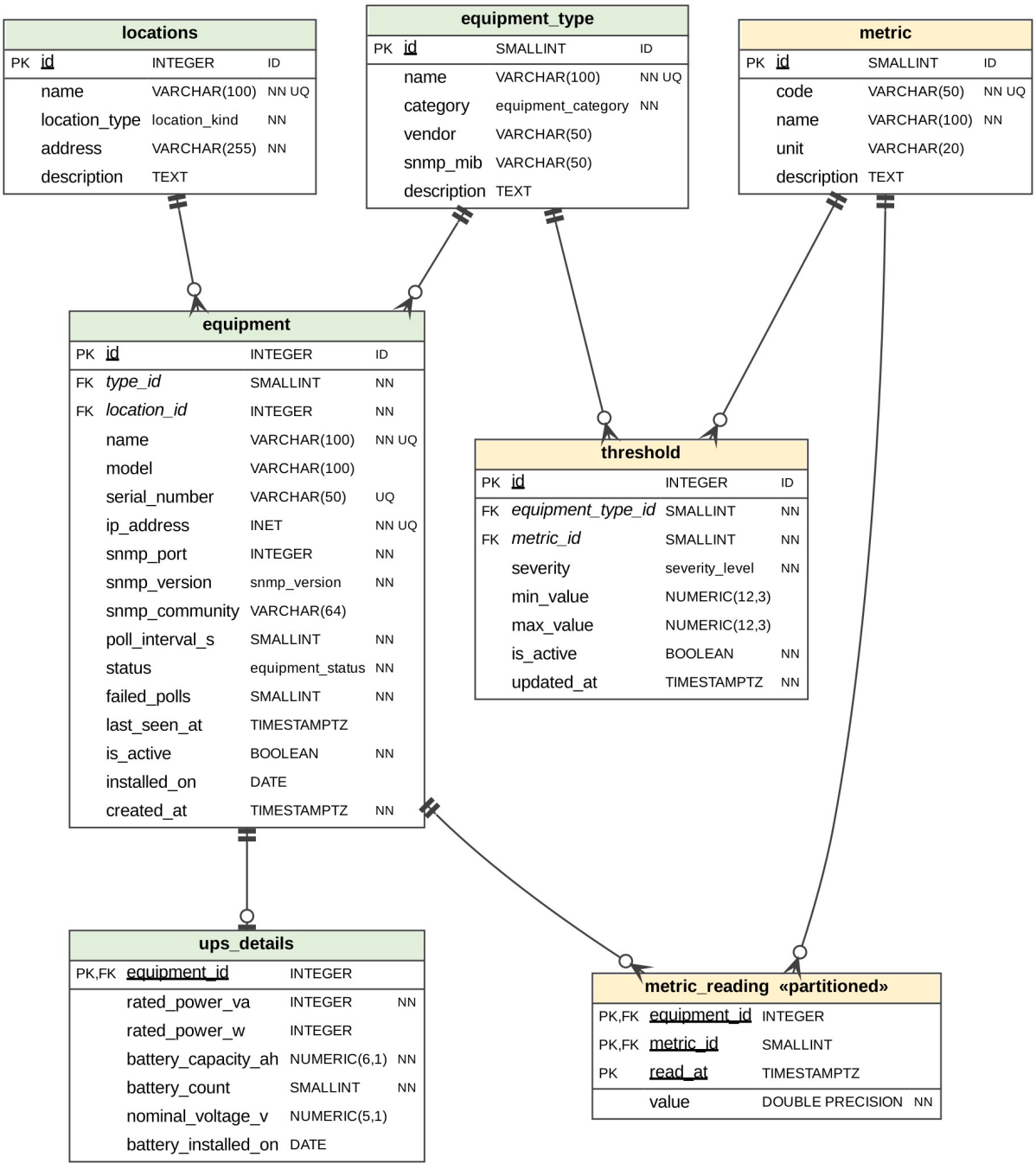


Рисунок 4 — Фізична модель: обладнання та телеметрія

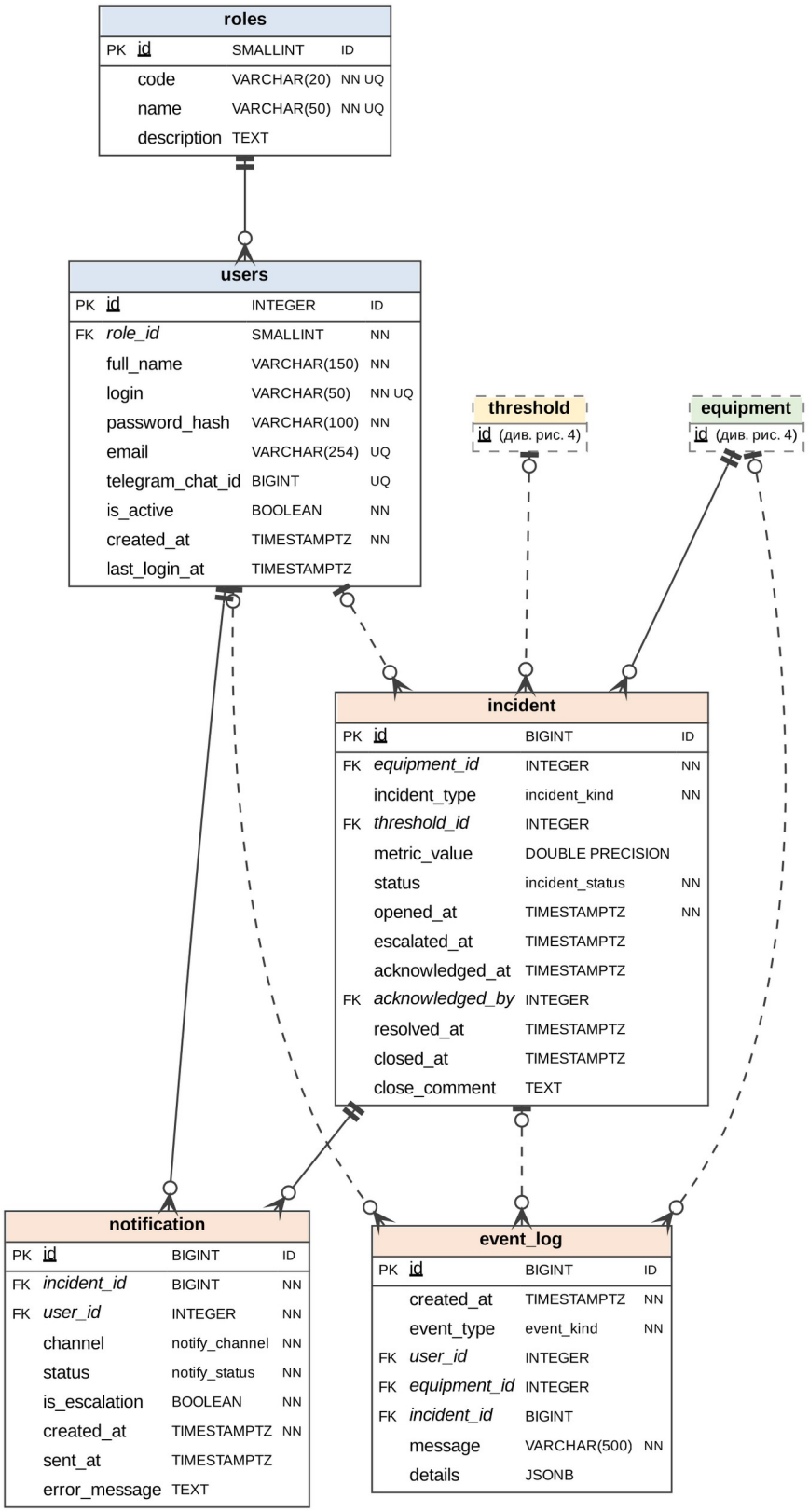


Рисунок 5 — Фізична модель: користувачі, інциденти, сповіщення

5.3 Типи даних

Типи даних обрано за принципом мінімального достатнього розміру: це особливо важливо для таблиці `metric_reading`, де кожен зайвий байт помножується на мільярди рядків. Обґрунтування наведено в табл. 8.

Таблиця 8 — Використані типи даних PostgreSQL

Тип	Розмір	Де використано	Обґрунтування
SMALLINT	2 Б	`id` довідників `roles`, `equipment_type`, `metric`; `poll_interval_s`, `failed_polls`, `battery_count`	Діапазон до 32 767 достатній; `metric_id` у кожному рядку телеметрії
INTEGER	4 Б	`id` таблиць `users`, `locations`, `equipment`, `threshold`; `snmp_port`; потужності ДБЖ	Діапазон до $2{,}1 \cdot 10^9$; порт до 65 535 не вміщується в SMALLINT
BIGINT	8 Б	`id` таблиць `incident`, `notification`, `event_log`; `telegram_chat_id`	Таблиці, що постійно ростуть; ідентифікатори Telegram виходять за межі INTEGER
GENERATED ALWAYS AS IDENTITY	—	Сурогатні ключі всіх таблиць, крім `metric_reading`, `ups_details`	Стандарт SQL замість SERIAL; забороняє довільне ручне задання ключа
VARCHAR(n)	n + 1…4 Б	Назви, коди, логін, e-mail, адреса, `message`	Довжина — частина домену (e-mail ≤ 254 за RFC 5321)
TEXT	змінний	`description`, `close_comment`, `error_message`	Довільний текст без обмеження довжини
NUMERIC(p,s)	змінний	`min_value`, `max_value` (12,3); ємність АКБ (6,1); напруга (5,1)	Точні десяткові значення, введені людиною, без похибки двійкового представлення
DOUBLE PRECISION	8 Б	`metric_reading.value`, `incident.metric_value`	Виміряні значення; фіксований розмір і швидка арифметика. REAL (4 Б, 6–7 значущих цифр) відхилено: uptime у секундах перевищує $16{,}7 \cdot 10^6$
BOOLEAN	1 Б	`is_active`, `is_escalation`	Логічні ознаки
DATE	4 Б	`installed_on`, `battery_installed_on`	Календарні дати без часу
TIMESTAMPTZ	8 Б	Усі моменти часу (`read_at`, `opened_at`, `created_at`…)	Зберігається в UTC; коректна робота з часовими поясами й переходом на літній час
INET	7 / 19 Б	`equipment.ip_address`	Перевірка формату IPv4/IPv6, оператори підмереж (<<=) для фільтрації
JSONB	змінний	`event_log.details`	Довільна структура «було/стало», можливість GIN-індексу
ENUM (10 типів)	4 Б	Статуси, категорії, канали (табл. 9)	Контроль допустимих значень, порядок значень за життєвим циклом

5.4 Перелічувані типи

Для фіксованих наборів значень, з якими пов’язана логіка програмного коду (статуси діаграми станів, канали доставки), використано перелічувані типи ENUM (табл. 9). Переліки, які змінює адміністратор (ролі, типи обладнання, метрики), оформлено окремими таблицями-довідниками, оскільки їх поповнення не повинно потребувати зміни схеми.

Таблиця 9 — Перелічувані типи (ENUM)

Тип	Допустимі значення	Використання
`location_kind`	data_center, access_node	`locations.location_type`

Тип	Допустимі значення	Використання
equipment_category	switch, router, onu, ups	equipment_type.category
equipment_status	unknown, online, offline, maintenance	equipment.status
snmp_version	v1, v2c, v3	equipment.snmp_version
severity_level	warning, critical	threshold.severity
incident_kind	threshold, offline	incident.incident_type
incident_status	new, escalated, acknowledged, resolved, closed	incident.status
notify_channel	web, email, telegram	notification.channel
notify_status	queued, sent, failed	notification.status
event_kind	status_change, incident, user_action, config_change, auth	event_log.event_type

5.5 Індекси

PostgreSQL автоматично створює індекси лише для первинних ключів та обмежень UNIQUE. Індекси для зовнішніх ключів і типових запитів створено явно (табл. 10). Часткові індекси (з умовою WHERE) охоплюють лише «гарячу» частину даних — відкриті інциденти, повідомлення в черзі — і тому залишаються малими.

Таблиця 10 — Індекси бази даних

Індекс	Таблиця (поля)	Вид	Призначення
pk_metric_reading	metric_reading (equipment_id, metric_id, read_at)	B-tree, PK	Історія показника для графіків (FR-12), останнє значення (FR-05)
ix_reading_read_at_brin	metric_reading (read_at)	BRIN	Звіти за період по всьому парку; розмір — кілобайти на мільйони рядків
ix_equipment_type, ix_equipment_location	equipment (type_id), (location_id)	B-tree	Зовнішні ключі; фільтр за типом і вузлом
ix_equipment_status	equipment (status) WHERE is_active	Частковий	Панель стану: пристрої в аварійному стані
ix_users_role, ix_threshold_metric	users (role_id), threshold (metric_id)	B-tree	Зовнішні ключі
ix_incident_equipment	incident (equipment_id, opened_at DESC)	B-tree	Історія інцидентів пристрою
ix_incident_opened	incident (opened_at)	B-tree	Звіти за період (FR-10)
ix_incident_new	incident (opened_at) WHERE status = 'new'	Частковий	Пошук інцидентів для ескалації (5 хв)
uq_incident_open	incident (equipment_id, incident_type, threshold_id) NULLS NOT DISTINCT WHERE status IN (new, escalated, acknowledged)	Частковий унікальний	БП-10: захист від дублювання відкритих інцидентів
ix_incident_threshold, ix_incident_ack_user	incident (threshold_id), (acknowledged_by)	Частковий	Зовнішні ключі (лише не-NULL значення)
ix_notification_antiflood	notification (incident_id, user_id, channel, created_at DESC)	B-tree	SR-07.2: перевірка, чи надсилалося сповіщення за 10 хв
ix_notification_queued	notification (created_at) WHERE status = 'queued'	Частковий	Черга відправлення Notification Service

Індекс	Таблиця (поля)	Вид	Призначення
ix_notification_user	notification (user_id)	B-tree	Зовнішній ключ
ix_event_log_created, ix_event_log_type, ix_event_log_equipment	event_log (created_at), (event_type, created_at), (equipment_id, created_at)	B-tree / частковий	FR-09: фільтрація журналу за датою, типом, вузлом
ix_event_log_user, ix_event_log_incident	event_log (user_id), (incident_id)	Частковий	Зовнішні ключі

5.6 Секціонування та оцінка обсягу даних

Таблиця metric_reading отримує по рядку на кожну метрику кожного пристрою щохвилини. Для оцінки обсягу на PostgreSQL 16 було згенеровано 720 000 рядків (100 пристроїв × 5 метрик × 1440 хвилин) і виміряно фактичний розмір: 52,2 байта на рядок у самій таблиці та 96,6 байта разом з індексами (індекс первинного ключа — близько 44 байт на рядок, BRIN — 24 КБ на весь набір). Результати розрахунку наведено в табл. 11.

Таблиця 11 — Оцінка обсягу таблиці metric_reading

Показник	Типовий парк	Максимум (NFR-01)
Кількість одиниць обладнання	100	500
Метрик на одиницю (у середньому)	5	5
Рядків за добу (опитування раз на 60 с)	720 000	3 600 000
Обсяг за добу (≈ 96,6 Б/рядок)	≈ 70 МБ	≈ 348 МБ
Обсяг за місяць (30 діб)	≈ 2,1 ГБ	≈ 10,4 ГБ
Обсяг за рік	≈ 25 ГБ	≈ 127 ГБ
Відповідність диску 100 ГБ (NFR-02)	Так	Ні — потрібна політика зберігання

Оскільки за максимального навантаження річна історія не вміщується на диск 100 ГБ, таблицю секціоновано за діапазонами read_at: одна секція на календарний місяць (metric_reading_2026_10 тощо), а рядки поза підготовленими діапазонами потрапляють у секцію metric_reading_default. Секціонування дає змогу:

- видаляти застарілі дані миттєво — командою DETACH PARTITION / DROP TABLE для цілої секції замість повільного масового DELETE, що спричиняє «роздування» таблиці;
- архівувати від’єднані секції (pg_dump) і зберігати на сервері 12 місяців історії для типового парку або 6 місяців (≈ 64 ГБ) за максимального навантаження;
- відсікати непотрібні секції під час виконання запитів за період (partition pruning), зменшуючи обсяг читання для звітів і графіків.

Скрипт створює секції від трьох місяців до поточного до дванадцяти місяців наперед; створення наступних секцій і видалення старих автоматизує процедура обслуговування, яку буде розроблено в лабораторній роботі №2.

5.7 Безпека та збереження даних

- Паролі зберігаються лише як bcrypt-хеші (users.password_hash); відкритий пароль у БД не потрапляє.
- Доступ до БД розмежовується ролями СУБД за принципом найменших привілеїв: окремі облікові записи для кожного сервісу та ролі «адміністратор», «сапорт»,

«звіти» (лише читання); рядок спільноти SNMP (`snmp_community`) доступний лише сервісу опитування й адміністратору — правами на рівні стовпця.

– Дії, що змінюють конфігурацію, фіксуються в `event_log` з ініціатором і значеннями «було/стало».
– Резервне копіювання — щоденний `pg_dump` схеми `netguard` та архів від'єднаних секцій телеметрії.

6. Опис таблиць бази даних

Нижче описано всі 12 таблиць БД. Для кожного поля зазначено тип даних PostgreSQL, участь у ключах (PK — первинний, FK — зовнішній, UQ — унікальний), допустимість NULL, значення за замовчуванням, правило видалення для зовнішніх ключів і обмеження CHECK. Типи й обмеження в таблицях звіту отримано безпосередньо з системного каталогу PostgreSQL після виконання DDL-скрипту, тому вони точно відповідають фізичній моделі.

6.1 Таблиця roles

Довідник ролей доступу. Містить дві ролі — «Сапорт» (`support`) та «Адміністратор» (`admin`); права ролі перевіряються застосунком (API Gateway) і дублюються ролями СУБД.

Таблиця 12 — Структура таблиці roles

Поле	Тип даних	Ключ	NULL	Опис, обмеження
`id`	SMALLINT	PK	ні	Ідентифікатор ролі (сурогатний ключ). GENERATED ALWAYS AS IDENTITY.
`code`	VARCHAR(20)	UQ	ні	Системний код ролі, на який спирається код застосунку (`support`, `admin`).
`name`	VARCHAR(50)	UQ	ні	Назва ролі для інтерфейсу.
`description`	TEXT	—	так	Опис повноважень ролі.

6.2 Таблиця users

Облікові записи інженерів і адміністраторів (FR-01, FR-11). Пароль у відкритому вигляді не зберігається. Фізичне видалення користувача, який підтверджував інциденти або отримував сповіщення, заборонено (RESTRICT) — доступ блокується ознакою `is_active`.

Таблиця 13 — Структура таблиці users

Поле	Тип даних	Ключ	NULL	Опис, обмеження
`id`	INTEGER	PK	ні	Ідентифікатор користувача. GENERATED ALWAYS AS IDENTITY.
`role_id`	SMALLINT	FK	ні	Роль користувача. → roles.id (ON DELETE RESTRICT).
`full_name`	VARCHAR(150)	—	ні	Прізвище, ім'я, по батькові.
`login`	VARCHAR(50)	UQ	ні	Логін для входу. CHECK: 3–50 символів, малі латинські літери, цифри, «_», «.».
`password_hash`	VARCHAR(100)	—	ні	Хеш пароля bcrypt (60 символів; запас довжини — для зміни алгоритму, напр. Argon2).
`email`	VARCHAR(254)	UQ	так	Адреса для e-mail-сповіщень. CHECK: формат «ім'я@домен.зона»; довжина 254 — максимум за RFC 5321.
`telegram_chat_id`	BIGINT	UQ	так	Ідентифікатор чату Telegram-бота для сповіщень.

Поле	Тип даних	Ключ	NULL	Опис, обмеження
`is_active`	BOOLEAN	—	ні	Ознака активного облікового запису; FALSE — доступ заблоковано. За замовч. TRUE.
`created_at`	TIMESTAMPTZ	—	ні	Момент створення облікового запису. За замовч. now().
`last_login_at`	TIMESTAMPTZ	—	так	Момент останнього успішного входу.

6.3 Таблиця locations

Вузли провайдера, де встановлено обладнання: ЦОД і районні вузли доступу.

Таблиця 14 — Структура таблиці locations

Поле	Тип даних	Ключ	NULL	Опис, обмеження
`id`	INTEGER	PK	ні	Ідентифікатор вузла. GENERATED ALWAYS AS IDENTITY.
`name`	VARCHAR(100)	UQ	ні	Назва вузла (наприклад, «ЦОД», «Вузол Оболонь»).
`location_type`	location_kind	—	ні	Тип вузла: `data_center` — ЦОД, `access_node` — районний вузол доступу.
`address`	VARCHAR(255)	—	ні	Поштова адреса вузла.
`description`	TEXT	—	так	Додаткові відомості (поверх, шафа, контакти).

6.4 Таблиця equipment_type

Довідник типів (моделей) обладнання. Тип визначає профіль SNMP-опитування і набір порогових значень, тому нове обладнання підключається без зміни структури БД (вимога масштабованості).

Таблиця 15 — Структура таблиці equipment_type

Поле	Тип даних	Ключ	NULL	Опис, обмеження
`id`	SMALLINT	PK	ні	Ідентифікатор типу. GENERATED ALWAYS AS IDENTITY.
`name`	VARCHAR(100)	UQ	ні	Назва типу/моделі (наприклад, «ДБЖ APC Symmetra PX»).
`category`	equipment_category	—	ні	Категорія: `switch`, `router`, `onu`, `ups`.
`vendor`	VARCHAR(50)	—	так	Виробник.
`snmp_mib`	VARCHAR(50)	—	так	База MIB, за якою опитується тип (UPS-MIB, PowerNet-MIB, IF-MIB…).
`description`	TEXT	—	так	Опис типу.

6.5 Таблиця metric

Довідник метрик телеметрії. Винесення метрик в окрему таблицю усуває дублювання назв у таблицях порогів і показників (ЗНФ) та дозволяє зберігати одиницю виміру.

Таблиця 16 — Структура таблиці metric

Поле	Тип даних	Ключ	NULL	Опис, обмеження
`id`	SMALLINT	PK	ні	Ідентифікатор метрики; SMALLINT зменшує розмір рядка в `metric_reading`. GENERATED ALWAYS AS IDENTITY.
`code`	VARCHAR(50)	UQ	ні	Системний код (`battery_charge`, `temperature`, `input_voltage`…). CHECK: `^[a-z][a-z0-9_]*$`.
`name`	VARCHAR(100)	—	ні	Назва для інтерфейсу (наприклад, «Заряд АКБ»).
`unit`	VARCHAR(20)	—	так	Одиниця виміру: %, °C, В, с тощо.
`description`	TEXT	—	так	Опис метрики, джерело (OID).

6.6 Таблиця equipment

Центральна таблиця: одиниці мережевого обладнання та ДБЖ, які опитує Polling Service (FR-02). Поля `status`, `failed_polls`, `last_seen_at` зберігають оперативний стан пристрою для панелі моніторингу (FR-05) і правила SR-04.2. Обладнання з історією інцидентів не видаляється фізично, а виводиться з експлуатації (`is_active = FALSE`).

Таблиця 17 — Структура таблиці equipment

Поле	Тип даних	Ключ	NULL	Опис, обмеження
`id`	INTEGER	PK	ні	Ідентифікатор одиниці обладнання. GENERATED ALWAYS AS IDENTITY.
`type_id`	SMALLINT	FK	ні	Тип (модель) обладнання. → equipment_type.id (ON DELETE RESTRICT).
`location_id`	INTEGER	FK	ні	Вузол, де встановлено пристрій. → locations.id (ON DELETE RESTRICT).
`name`	VARCHAR(100)	UQ	ні	Мережеве ім’я пристрою (hostname).
`model`	VARCHAR(100)	—	так	Модель за паспортом.
`serial_number`	VARCHAR(50)	UQ	так	Серійний номер.
`ip_address`	INET	UQ	ні	IP-адреса для SNMP-опитування (IPv4/IPv6). CHECK: адреса вузла, а не мережі (маска /32 або /128).
`snmp_port`	INTEGER	—	ні	UDP-порт агента SNMP. CHECK: 1–65535 (тому INTEGER, а не SMALLINT з максимумом 32767). За замовч. 161.
`snmp_version`	snmp_version	—	ні	Версія протоколу SNMP. За замовч. 'v2c'.
`snmp_community`	VARCHAR(64)	—	так	Рядок спільноти SNMP v2c (секрет; доступ до стовпця обмежується правами).
`poll_interval_s`	SMALLINT	—	ні	Інтервал опитування, с. CHECK: 10–60 (FR-04: не рідше ніж раз на 60 с). За замовч. 60.
`status`	equipment_status	—	ні	Оперативний стан: `unknown`, `online`, `offline`, `maintenance`. За замовч. 'unknown'.
`failed_polls`	SMALLINT	—	ні	Кількість поспіль невдалих опитувань; ≥ 3 — перехід у `offline` (SR-04.2). CHECK: ≥ 0. За замовч. 0.
`last_seen_at`	TIMESTAMPTZ	—	так	Момент останньої успішної відповіді пристрою.

Поле	Тип даних	Ключ	NULL	Опис, обмеження
is_active	BOOLEAN	—	ні	Ознака експлуатації; FALSE — пристрій виведено з експлуатації (не опитується). За замовч. TRUE.
installed_on	DATE	—	так	Дата встановлення.
created_at	TIMESTAMPTZ	—	ні	Момент реєстрації в системі. За замовч. now().

6.7 Таблиця ups_details

Паспортні характеристики ДБЖ (FR-03) — підтип таблиці equipment у зв’язку 1:1 зі спільним первинним ключем. Рядок існує лише для обладнання категорії ups; при видаленні обладнання видаляється каскадно.

Таблиця 18 — Структура таблиці ups_details

Поле	Тип даних	Ключ	NULL	Опис, обмеження
equipment_id	INTEGER	PK,FK	ні	Одиниця обладнання, для якої задано характеристики ДБЖ. → equipment.id (ON DELETE CASCADE).
rated_power_va	INTEGER	—	ні	Номінальна повна потужність, В·А. CHECK: > 0.
rated_power_w	INTEGER	—	так	Номінальна активна потужність, Вт. CHECK: від 1 до rated_power_va (коефіцієнт потужності ≤ 1).
battery_capacity_ah	NUMERIC(6,1)	—	ні	Номінальна ємність АКБ, А·год. CHECK: > 0.
battery_count	SMALLINT	—	ні	Кількість батарей (батарейних модулів). CHECK: > 0. За замовч. 1.
nominal_voltage_v	NUMERIC(5,1)	—	так	Номінальна напруга батарейної шини, В. CHECK: > 0.
battery_installed_on	DATE	—	так	Дата встановлення АКБ — для планування заміни.

6.8 Таблиця threshold

Порогові значення метрик для типу обладнання (FR-06). Значення поза діапазоном [min_value; max_value] вважається порушенням; одна з меж може бути відсутня (наприклад, лише «заряд АКБ < 30 %»).

Таблиця 19 — Структура таблиці threshold

Поле	Тип даних	Ключ	NULL	Опис, обмеження
id	INTEGER	PK	ні	Ідентифікатор порогу. GENERATED ALWAYS AS IDENTITY.
equipment_type_id	SMALLINT	FK	ні	Тип обладнання, для якого діє поріг. → equipment_type.id (ON DELETE CASCADE).
metric_id	SMALLINT	FK	ні	Метрика, що контролюється. → metric.id (ON DELETE RESTRICT).
severity	severity_level	—	ні	Рівень критичності: warning, critical. За замовч. 'warning'.

Поле	Тип даних	Ключ	NULL	Опис, обмеження
min_value	NUMERIC(12,3)	—	так	Нижня межа норми (значення < min_value — порушення).
max_value	NUMERIC(12,3)	—	так	Верхня межа норми (значення > max_value — порушення).
is_active	BOOLEAN	—	ні	Ознака дії порогу; поріг, на який посилаються інциденти, деактивується замість видалення. За замовч. TRUE.
updated_at	TIMESTAMPTZ	—	ні	Момент останньої зміни (автор зміни фіксується в event_log). За замовч. now().

Обмеження рівня таблиці та індекси:

- uq_threshold UNIQUE (equipment_type_id, metric_id, severity) — один поріг на рівень критичності;
- ck_threshold_bounds — задано хоча б одну межу; ck_threshold_range — min_value < max_value.

6.9 Таблиця metric_reading

Історія показників телеметрії (FR-04, FR-12) — найбільша таблиця БД. Секціонована за діапазонами read_at (одна секція на місяць). Первинний ключ — природний складений.

Таблиця 20 — Структура таблиці metric_reading

Поле	Тип даних	Ключ	NULL	Опис, обмеження
equipment_id	INTEGER	PK,FK	ні	Пристрій, з якого знято показник. → equipment.id (ON DELETE CASCADE).
metric_id	SMALLINT	PK,FK	ні	Метрика. → metric.id (ON DELETE RESTRICT).
read_at	TIMESTAMPTZ	PK	ні	Момент зняття показника; ключ секціонування.
value	DOUBLE PRECISION	—	ні	Значення показника. CHECK: скінченне число (не NaN, не ±Infinity).

Обмеження рівня таблиці та індекси:

- Секції metric_reading_PPPP_MM + metric_reading_default; індекс BRIN ix_reading_read_at_brin за read_at.

6.10 Таблиця incident

Інциденти (аварії) та їхній життєвий цикл (FR-07, FR-08, діаграма станів ЛР4 ТСПП). Статус узгоджено з моментами переходів обмеженнями CHECK.

Таблиця 21 — Структура таблиці incident

Поле	Тип даних	Ключ	NULL	Опис, обмеження
id	BIGINT	PK	ні	Номер інциденту. GENERATED ALWAYS AS IDENTITY.
equipment_id	INTEGER	FK	ні	Пристрій, на якому виникла аварія. → equipment.id (ON DELETE RESTRICT).
incident_type	incident_kind	—	ні	Тип: threshold — порушення порогу, offline — недоступність пристрою.
threshold_id	INTEGER	FK	так	Порушений поріг; обов’язковий для типу threshold, NULL — для offline. → threshold.id (ON DELETE RESTRICT).

Поле	Тип даних	Ключ	NULL	Опис, обмеження
metric_value	DOUBLE PRECISION	—	так	Значення показника на момент відкриття (знімок); обов’язкове для типу threshold.
status	incident_status	—	ні	Статус: new, escalated, acknowledged, resolved, closed. За замовч. 'new'.
opened_at	TIMESTAMPTZ	—	ні	Момент виявлення. За замовч. now().
escalated_at	TIMESTAMPTZ	—	так	Момент ескалації адміністратору (якщо інцидент не прийнято за 5 хв).
acknowledged_at	TIMESTAMPTZ	—	так	Момент підтвердження (acknowledge).
acknowledged_by	INTEGER	FK	так	Користувач, що підтвердив інцидент. → users.id (ON DELETE RESTRICT).
resolved_at	TIMESTAMPTZ	—	так	Момент усунення несправності.
closed_at	TIMESTAMPTZ	—	так	Момент закриття.
close_comment	TEXT	—	так	Коментар щодо вжитих заходів; обов’язковий для статусу closed.

Обмеження рівня таблиці та індекси:

- ck_incident_threshold, ck_incident_value, ck_incident_escalated, ck_incident_ack, ck_incident_resolved, ck_incident_closed, ck_incident_time_order — узгодженість типу, статусу та часу;
- uq_incident_open — частковий унікальний індекс: не більше одного відкритого інциденту на (обладнання, тип, поріг).

6.11 Таблиця notification

Сповіщення про інциденти: один рядок — одне повідомлення одному користувачу одним каналом (FR-07, SR-07.1). Історія відправлень потрібна для антифлуд-логіки (SR-07.2) та ескалації.

Таблиця 22 — Структура таблиці notification

Поле	Тип даних	Ключ	NULL	Опис, обмеження
id	BIGINT	PK	ні	Ідентифікатор сповіщення. GENERATED ALWAYS AS IDENTITY.
incident_id	BIGINT	FK	ні	Інцидент, про який сповіщено. → incident.id (ON DELETE CASCADE).
user_id	INTEGER	FK	ні	Отримувач. → users.id (ON DELETE RESTRICT).
channel	notify_channel	—	ні	Канал доставки: web, email, telegram.
status	notify_status	—	ні	Стан доставки: queued, sent, failed. За замовч. 'queued'.
is_escalation	BOOLEAN	—	ні	Ознака ескалаційного сповіщення адміністратору. За замовч. FALSE.
created_at	TIMESTAMPTZ	—	ні	Момент постановки в чергу. За замовч. now().
sent_at	TIMESTAMPTZ	—	так	Момент відправлення; обов’язковий для sent, не раніше created_at.
error_message	TEXT	—	так	Текст помилки доставки; обов’язковий для failed.

6.12 Таблиця event_log

Журнал подій (FR-09): зміни стану обладнання, інциденти, дії користувачів, зміни конфігурації та автентифікація. Запис журналу зберігається і після видалення пов’язаних об’єктів (SET NULL).

Таблиця 23 — Структура таблиці event_log

Поле	Тип даних	Ключ	NULL	Опис, обмеження
`id`	BIGINT	PK	ні	Ідентифікатор запису. GENERATED ALWAYS AS IDENTITY.
`created_at`	TIMESTAMPTZ	—	ні	Момент події. За замовч. now().
`event_type`	event_kind	—	ні	Тип: `status_change`, `incident`, `user_action`, `config_change`, `auth`.
`user_id`	INTEGER	FK	так	Користувач-ініціатор (NULL — системна подія). → users.id (ON DELETE SET NULL).
`equipment_id`	INTEGER	FK	так	Пристрій, якого стосується подія. → equipment.id (ON DELETE SET NULL).
`incident_id`	BIGINT	FK	так	Інцидент, якого стосується подія. → incident.id (ON DELETE SET NULL).
`message`	VARCHAR(500)	—	ні	Короткий опис події.
`details`	JSONB	—	так	Деталі у форматі JSON (наприклад, значення «було/стало»). CHECK: JSON-об’єкт.

7. Висновки

У лабораторній роботі спроєктовано реляційну базу даних системи моніторингу мережевого обладнання та ДБЖ інтернет-провайдера NetGuard Monitor на трьох рівнях абстракції.

Контекстна модель визначила межі БД, п’ять сервісів-клієнтів і потоки даних між ними, а також 12 сутностей предметної області та 16 зв’язків між ними (ER-діаграма в нотації Чена). Порівняно з попередніми роботами модель доповнено довідником метрик, підтипом «ДБЖ», сповіщеннями та журналом подій, а інцидент — типом «офлайн» і повним життєвим циклом.

Логічна модель містить 12 відношень, нормалізованих до НФБК, з визначеними первинними, альтернативними та зовнішніми ключами, правилами референційної цілісності та 14 бізнес-правилами, які зіставлено з механізмами їх забезпечення. Контрольовану денормалізацію (знімок значення в інциденті, оперативний стан обладнання) обґрунтовано вимогами до продуктивності та збереження історії.

Фізична модель реалізована для PostgreSQL 15+: 10 перелічуваних типів, 12 таблиць, 66 обмежень (12 PK, 16 FK, 12 UNIQUE, 26 CHECK) і 20 явно створених індексів, серед яких BRIN, часткові та частковий унікальний. Типи даних обрано з урахуванням обсягу телеметрії. За виміряним розміром рядка (≈ 97 байт) показано, що за максимального навантаження річна історія займає ≈ 127 ГБ, тому таблицю показників секціоновано за місяцями з політикою зберігання. DDL-скрипт перевірено на PostgreSQL 16.

За результатами проєктування сформульовано функціональні вимоги до програмної реалізації БД, яку буде виконано й проілюстровано в лабораторній роботі №2 (табл. 24).

Таблиця 24 — Функціональні вимоги до програмної реалізації БД (ЛР2)

Код	Функціональна вимога	Об’єкти БД / спосіб реалізації	Джерело
ФВ-01	Створення БД: схема, перелічувані типи, таблиці, обмеження, індекси й коментарі розгортаються одним повторно виконуваним скриптом	`sql/schema.sql`	Усі FR
ФВ-02	Наповнення тестовими даними: ≥ 2 ролей, ≥ 5 користувачів, ≥ 4 вузлів, ≥ 5 типів, ≥ 6 метрик, ≥ 20 пристроїв (з них ≥ 5 ДБЖ), ≥ 10 порогів, показники за ≥ 7 діб, ≥ 15 інцидентів у різних статусах, сповіщення, журнал	Скрипт `seed.sql`, функції `generate_series`	Критерії прийняття SRS
ФВ-03	Автентифікація і керування користувачами: перевірка логіна та bcrypt-хешу, відмова заблокованим, фіксація входу; створення, зміна ролі, блокування користувачів	Функція `fn_authenticate` (розширення `pgcrypto`), `users.last_login_at`, `event_log` (auth)	FR-01, FR-11
ФВ-04	Розмежування доступу на рівні СУБД за принципом найменших привілеїв	Ролі `ng_admin`, `ng_support`, `ng_poller`, `ng_alerting`, `ng_notifier`, `ng_report`; GRANT на таблиці та стовпці	FR-01, безпека
ФВ-05	Реєстрація, зміна та виведення з експлуатації обладнання; реєстрація ДБЖ однією транзакцією; заборона паспорта ДБЖ для обладнання іншої категорії	Процедура `sp_register_ups`, тригер на `ups_details`, `equipment.is_active`	FR-02, FR-03, БП-06
ФВ-06	Пакетний запис телеметрії з ігноруванням дублікатів і оновленням оперативного стану пристрою	INSERT … ON CONFLICT DO NOTHING / COPY; оновлення `status`, `last_seen_at`, `failed_polls`	FR-04, SR-04.1
ФВ-07	Облік недоступності: після 3 поспіль невдалих опитувань — статус «офлайн», інцидент типу `offline` і запис журналу	Процедура `sp_register_poll_failure`	SR-04.2
ФВ-08	Автоматична перевірка порогів: при надходженні показника — порівняння з активними порогами типу і створення інциденту без дублювання відкритих	Тригер AFTER INSERT на `metric_reading` / функція `fn_check_thresholds`; `uq_incident_open`	FR-06, FR-07, БП-10
ФВ-09	Відображення поточного стану: останнє значення кожної метрики, статус, вузол, кількість відкритих інцидентів	Представлення `v_equipment_current_state` (DISTINCT ON / LATERAL)	FR-05
ФВ-10	Керування пороговими значеннями з фіксацією автора та значень «було/стало»	CRUD `threshold`, тригер аудиту → `event_log` (config_change)	FR-06, FR-09
ФВ-11	Життєвий цикл інциденту лише за допустимими переходами діаграми станів	Процедури `sp_acknowledge_incident`, `sp_resolve_incident`, `sp_close_incident`; тригер перевірки переходів	FR-08, БП-11
ФВ-12	Ескалація: інциденти в статусі `new`, старші 5 хв, переводяться в `escalated` зі сповіщенням адміністраторів	Функція `fn_escalate_overdue`, індекс `ix_incident_new`	Сценарій UC «Отримання сповіщень»
ФВ-13	Постановка сповіщень у чергу з антифлуд-перевіркою (не частіше одного за 10 хв) та оновлення стану доставки	Функція `fn_enqueue_notification`, `ix_notification_antiflood`, `ix_notification_queued`	FR-07, SR-07.1, SR-07.2

Код	Функціональна вимога	Об’єкти БД / спосіб реалізації	Джерело
ФВ-14	Журнал подій: автоматичний запис змін обладнання, порогів, користувачів та інцидентів; фільтрація за періодом, вузлом і типом	Тригери аудиту, запит/функція `fn_event_log_search`	FR-09
ФВ-15	Історія показника за період з агрегуванням для графіків	Функція `fn_metric_history` (`date_bin`, avg/min/max)	FR-12
ФВ-16	Звіти за період: uptime обладнання, кількість і середня тривалість інцидентів (MTTA, MTTR) за вузлами й типами, стан АКБ ДБЖ; вивантаження у CSV	Представлення `v_report_*`, команда COPY	FR-10, SR-10.1
ФВ-17	Обслуговування секцій телеметрії: створення секцій наперед, від’єднання й архівування застарілих	Процедури `sp_create_partitions`, `sp_drop_old_partitions`	NFR-01, NFR-02
ФВ-18	Резервне копіювання та відновлення БД	Скрипти `pg_dump` / `pg_restore`, перевірка відновлення	Надійність
ФВ-19	Демонстрація цілісності: набір запитів, що показують спрацювання PK, FK, UNIQUE, CHECK і відкат транзакції при помилці	Тестовий скрипт `tests.sql`	Критерії прийняття
ФВ-20	Перевірка продуктивності ключових запитів (поточний стан, історія, звіти) з використанням індексів	EXPLAIN (ANALYZE, BUFFERS); час відповіді < 2 с	NFR-01

Реалізація цих вимог у лабораторній роботі №2 перетворить спроєктовану модель на працездатну БД, яка самостійно забезпечує цілісність даних і ключові бізнес-правила системи (перевірка порогів, життєвий цикл інцидентів, антифлуд, журнал подій) незалежно від коду мікросервісів. Це відповідає критеріям прийняття SRS: не менше 6 взаємопов’язаних таблиць з тестовими даними, розмежування прав ролей «Сапорт» і «Адміністратор» та формування звітів.

Додаток А. DDL-скрипт фізичної моделі (sql/schema.sql)

```
-- ============================================================================
--  NetGuard Monitor — фізична модель бази даних
--  СУБД: PostgreSQL 15+ (перевірено на PostgreSQL 16)
--  Лабораторна робота №1, дисципліна «Організація баз даних»
--
--  Запуск:  psql -d netguard -v ON_ERROR_STOP=1 -f sql/schema.sql
-- ============================================================================

DROP SCHEMA IF EXISTS netguard CASCADE;
CREATE SCHEMA netguard;
SET search_path TO netguard;

-- ----------------------------------------------------------------------------
-- 1. Користувацькі перелічувані типи (домени зі сталим набором значень)
-- ----------------------------------------------------------------------------
CREATE TYPE location_kind      AS ENUM ('data_center', 'access_node');
CREATE TYPE equipment_category AS ENUM ('switch', 'router', 'onu', 'ups');
CREATE TYPE equipment_status   AS ENUM ('unknown', 'online', 'offline', 'maintenance');
CREATE TYPE snmp_version       AS ENUM ('v1', 'v2c', 'v3');
CREATE TYPE severity_level     AS ENUM ('warning', 'critical');
CREATE TYPE incident_kind      AS ENUM ('threshold', 'offline');
CREATE TYPE incident_status    AS ENUM ('new', 'escalated', 'acknowledged', 'resolved', 'closed');
CREATE TYPE notify_channel     AS ENUM ('web', 'email', 'telegram');
CREATE TYPE notify_status      AS ENUM ('queued', 'sent', 'failed');
CREATE TYPE event_kind         AS ENUM ('status_change', 'incident', 'user_action', 'config_change',
'auth');

-- ----------------------------------------------------------------------------
-- 2. Довідники та облікові записи
-- ----------------------------------------------------------------------------
CREATE TABLE roles (
    id          SMALLINT     GENERATED ALWAYS AS IDENTITY,
    code        VARCHAR(20)  NOT NULL,
    name        VARCHAR(50)  NOT NULL,
    description TEXT,
    CONSTRAINT pk_roles      PRIMARY KEY (id),
    CONSTRAINT uq_roles_code UNIQUE (code),
    CONSTRAINT uq_roles_name UNIQUE (name)
);

CREATE TABLE users (
    id               INTEGER      GENERATED ALWAYS AS IDENTITY,
    role_id          SMALLINT     NOT NULL,
    full_name        VARCHAR(150) NOT NULL,
    login            VARCHAR(50)  NOT NULL,
    password_hash    VARCHAR(100) NOT NULL,
    email            VARCHAR(254),
    telegram_chat_id BIGINT,
    is_active        BOOLEAN      NOT NULL DEFAULT TRUE,
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    last_login_at    TIMESTAMPTZ,
    CONSTRAINT pk_users          PRIMARY KEY (id),
    CONSTRAINT fk_users_role     FOREIGN KEY (role_id) REFERENCES roles (id)
                                 ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT uq_users_login    UNIQUE (login),
    CONSTRAINT uq_users_email    UNIQUE (email),
    CONSTRAINT uq_users_telegram UNIQUE (telegram_chat_id),
    CONSTRAINT ck_users_login    CHECK (login ~ '^[a-z0-9_.]{3,50}$'),
    CONSTRAINT ck_users_email    CHECK (email IS NULL OR email ~* '^[^@[:space:]]+@[^@[:space:]]+\.
[^@[:space:]]+$')
);
```

```
CREATE TABLE locations (
    id            INTEGER       GENERATED ALWAYS AS IDENTITY,
    name          VARCHAR(100)  NOT NULL,
    location_type location_kind NOT NULL,
    address       VARCHAR(255)  NOT NULL,
    description   TEXT,
    CONSTRAINT pk_locations      PRIMARY KEY (id),
    CONSTRAINT uq_locations_name UNIQUE (name)
);

CREATE TABLE equipment_type (
    id          SMALLINT           GENERATED ALWAYS AS IDENTITY,
    name        VARCHAR(100)       NOT NULL,
    category    equipment_category NOT NULL,
    vendor      VARCHAR(50),
    snmp_mib    VARCHAR(50),
    description TEXT,
    CONSTRAINT pk_equipment_type      PRIMARY KEY (id),
    CONSTRAINT uq_equipment_type_name UNIQUE (name)
);

CREATE TABLE metric (
    id          SMALLINT     GENERATED ALWAYS AS IDENTITY,
    code        VARCHAR(50)  NOT NULL,
    name        VARCHAR(100) NOT NULL,
    unit        VARCHAR(20),
    description TEXT,
    CONSTRAINT pk_metric      PRIMARY KEY (id),
    CONSTRAINT uq_metric_code UNIQUE (code),
    CONSTRAINT ck_metric_code CHECK (code ~ '^[a-z][a-z0-9_]*$')
);

-- ---------------------------------------------------------------------
-- 3. Обладнання
-- ---------------------------------------------------------------------
CREATE TABLE equipment (
    id              INTEGER          GENERATED ALWAYS AS IDENTITY,
    type_id         SMALLINT         NOT NULL,
    location_id     INTEGER          NOT NULL,
    name            VARCHAR(100)     NOT NULL,
    model           VARCHAR(100),
    serial_number   VARCHAR(50),
    ip_address      INET             NOT NULL,
    snmp_port       INTEGER          NOT NULL DEFAULT 161,
    snmp_version    snmp_version     NOT NULL DEFAULT 'v2c',
    snmp_community  VARCHAR(64),
    poll_interval_s SMALLINT         NOT NULL DEFAULT 60,
    status          equipment_status NOT NULL DEFAULT 'unknown',
    failed_polls    SMALLINT         NOT NULL DEFAULT 0,
    last_seen_at    TIMESTAMPTZ,
    is_active       BOOLEAN          NOT NULL DEFAULT TRUE,
    installed_on    DATE,
    created_at      TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT pk_equipment          PRIMARY KEY (id),
    CONSTRAINT fk_equipment_type     FOREIGN KEY (type_id) REFERENCES equipment_type (id)
                                     ON DELETE RESTRICT,
    CONSTRAINT fk_equipment_location FOREIGN KEY (location_id) REFERENCES locations (id)
                                     ON DELETE RESTRICT,
    CONSTRAINT uq_equipment_name     UNIQUE (name),
    CONSTRAINT uq_equipment_serial   UNIQUE (serial_number),
    CONSTRAINT uq_equipment_ip       UNIQUE (ip_address),
```

```
    CONSTRAINT ck_equipment_ip_host  CHECK (masklen(ip_address) = CASE family(ip_address) WHEN 4 THEN
32 ELSE 128 END),
    CONSTRAINT ck_equipment_port     CHECK (snmp_port BETWEEN 1 AND 65535),
    CONSTRAINT ck_equipment_interval CHECK (poll_interval_s BETWEEN 10 AND 60),
    CONSTRAINT ck_equipment_failed   CHECK (failed_polls >= 0)
);

-- Підтип «ДБЖ» (зв'язок 1:1 зі спільним первинним ключем)
CREATE TABLE ups_details (
    equipment_id         INTEGER      NOT NULL,
    rated_power_va       INTEGER      NOT NULL,
    rated_power_w        INTEGER,
    battery_capacity_ah  NUMERIC(6,1) NOT NULL,
    battery_count        SMALLINT     NOT NULL DEFAULT 1,
    nominal_voltage_v    NUMERIC(5,1),
    battery_installed_on DATE,
    CONSTRAINT pk_ups_details     PRIMARY KEY (equipment_id),
    CONSTRAINT fk_ups_equipment   FOREIGN KEY (equipment_id) REFERENCES equipment (id)
                                  ON DELETE CASCADE,
    CONSTRAINT ck_ups_power_va    CHECK (rated_power_va > 0),
    CONSTRAINT ck_ups_power_w     CHECK (rated_power_w IS NULL OR rated_power_w BETWEEN 1 AND
rated_power_va),
    CONSTRAINT ck_ups_capacity    CHECK (battery_capacity_ah > 0),
    CONSTRAINT ck_ups_battery_cnt CHECK (battery_count > 0),
    CONSTRAINT ck_ups_voltage     CHECK (nominal_voltage_v IS NULL OR nominal_voltage_v > 0)
);

-- ---------------------------------------------------------------------
-- 4. Порогові значення
-- ---------------------------------------------------------------------
CREATE TABLE threshold (
    id                INTEGER        GENERATED ALWAYS AS IDENTITY,
    equipment_type_id SMALLINT       NOT NULL,
    metric_id         SMALLINT       NOT NULL,
    severity          severity_level NOT NULL DEFAULT 'warning',
    min_value         NUMERIC(12,3),
    max_value         NUMERIC(12,3),
    is_active         BOOLEAN        NOT NULL DEFAULT TRUE,
    updated_at        TIMESTAMPTZ    NOT NULL DEFAULT now(),
    CONSTRAINT pk_threshold         PRIMARY KEY (id),
    CONSTRAINT fk_threshold_type    FOREIGN KEY (equipment_type_id) REFERENCES equipment_type (id)
                                    ON DELETE CASCADE,
    CONSTRAINT fk_threshold_metric  FOREIGN KEY (metric_id) REFERENCES metric (id)
                                    ON DELETE RESTRICT,
    CONSTRAINT uq_threshold         UNIQUE (equipment_type_id, metric_id, severity),
    CONSTRAINT ck_threshold_bounds  CHECK (min_value IS NOT NULL OR max_value IS NOT NULL),
    CONSTRAINT ck_threshold_range   CHECK (min_value IS NULL OR max_value IS NULL OR min_value <
max_value)
);

-- ---------------------------------------------------------------------
-- 5. Телеметрія (секціонована за місяцями таблиця)
-- ---------------------------------------------------------------------
CREATE TABLE metric_reading (
    equipment_id INTEGER          NOT NULL,
    metric_id    SMALLINT         NOT NULL,
    read_at      TIMESTAMPTZ      NOT NULL,
    value        DOUBLE PRECISION NOT NULL,
    CONSTRAINT pk_metric_reading        PRIMARY KEY (equipment_id, metric_id, read_at),
    CONSTRAINT fk_reading_equipment     FOREIGN KEY (equipment_id) REFERENCES equipment (id)
                                        ON DELETE CASCADE,
    CONSTRAINT fk_reading_metric        FOREIGN KEY (metric_id) REFERENCES metric (id)
                                        ON DELETE RESTRICT,
```

```
    CONSTRAINT ck_reading_value_finite  CHECK (value NOT IN ('NaN', 'Infinity', '-Infinity'))
) PARTITION BY RANGE (read_at);

-- Секції: від трьох місяців до поточного до дванадцяти місяців наперед
-- (назва секції: metric_reading_PPPP_MM) + секція за замовчуванням.
DO $$
DECLARE
    m_start date := date_trunc('month', now())::date - INTERVAL '3 months';
    m_end   date;
BEGIN
    FOR i IN 0..15 LOOP
        m_end := m_start + INTERVAL '1 month';
        EXECUTE format(
            'CREATE TABLE %I PARTITION OF metric_reading FOR VALUES FROM (%L) TO (%L)',
            'metric_reading_' || to_char(m_start, 'YYYY_MM'), m_start, m_end);
        m_start := m_end;
    END LOOP;
END $$;

CREATE TABLE metric_reading_default PARTITION OF metric_reading DEFAULT;

-- ---------------------------------------------------------------------
-- 6. Інциденти, сповіщення, журнал подій
-- ---------------------------------------------------------------------
CREATE TABLE incident (
    id              BIGINT           GENERATED ALWAYS AS IDENTITY,
    equipment_id    INTEGER          NOT NULL,
    incident_type   incident_kind    NOT NULL,
    threshold_id    INTEGER,
    metric_value    DOUBLE PRECISION,
    status          incident_status  NOT NULL DEFAULT 'new',
    opened_at       TIMESTAMPTZ      NOT NULL DEFAULT now(),
    escalated_at    TIMESTAMPTZ,
    acknowledged_at TIMESTAMPTZ,
    acknowledged_by INTEGER,
    resolved_at     TIMESTAMPTZ,
    closed_at       TIMESTAMPTZ,
    close_comment   TEXT,
    CONSTRAINT pk_incident            PRIMARY KEY (id),
    CONSTRAINT fk_incident_equipment  FOREIGN KEY (equipment_id) REFERENCES equipment (id)
                                      ON DELETE RESTRICT,
    CONSTRAINT fk_incident_threshold  FOREIGN KEY (threshold_id) REFERENCES threshold (id)
                                      ON DELETE RESTRICT,
    CONSTRAINT fk_incident_ack_user   FOREIGN KEY (acknowledged_by) REFERENCES users (id)
                                      ON DELETE RESTRICT,
    -- інцидент типу 'threshold' завжди посилається на поріг, 'offline' — ніколи
    CONSTRAINT ck_incident_threshold  CHECK ((incident_type = 'threshold') = (threshold_id IS NOT
NULL)),
    CONSTRAINT ck_incident_value      CHECK (incident_type = 'offline' OR metric_value IS NOT NULL),
    -- узгодженість статусу з датами життєвого циклу (діаграма станів, ЛР4)
    CONSTRAINT ck_incident_escalated  CHECK (status <> 'escalated' OR escalated_at IS NOT NULL),
    CONSTRAINT ck_incident_ack        CHECK (status IN ('new', 'escalated')
                                             OR (acknowledged_at IS NOT NULL AND acknowledged_by IS NOT
NULL)),
    CONSTRAINT ck_incident_resolved   CHECK (status NOT IN ('resolved', 'closed') OR resolved_at IS NOT
NULL),
    CONSTRAINT ck_incident_closed     CHECK (status <> 'closed'
                                             OR (closed_at IS NOT NULL AND close_comment IS NOT NULL
                                                 AND btrim(close_comment) <> '')),
    CONSTRAINT ck_incident_time_order CHECK (
        (escalated_at    IS NULL OR escalated_at    >= opened_at) AND
        (acknowledged_at IS NULL OR acknowledged_at >= opened_at) AND
        (resolved_at     IS NULL OR resolved_at     >= acknowledged_at) AND
```

```
        (closed_at       IS NULL OR closed_at       >= resolved_at))
);

CREATE TABLE notification (
    id            BIGINT         GENERATED ALWAYS AS IDENTITY,
    incident_id   BIGINT         NOT NULL,
    user_id       INTEGER        NOT NULL,
    channel       notify_channel NOT NULL,
    status        notify_status  NOT NULL DEFAULT 'queued',
    is_escalation BOOLEAN        NOT NULL DEFAULT FALSE,
    created_at    TIMESTAMPTZ    NOT NULL DEFAULT now(),
    sent_at       TIMESTAMPTZ,
    error_message TEXT,
    CONSTRAINT pk_notification          PRIMARY KEY (id),
    CONSTRAINT fk_notification_incident FOREIGN KEY (incident_id) REFERENCES incident (id)
                                        ON DELETE CASCADE,
    CONSTRAINT fk_notification_user     FOREIGN KEY (user_id) REFERENCES users (id)
                                        ON DELETE RESTRICT,
    CONSTRAINT ck_notification_sent     CHECK (status <> 'sent' OR sent_at IS NOT NULL),
    CONSTRAINT ck_notification_failed   CHECK (status <> 'failed' OR error_message IS NOT NULL),
    CONSTRAINT ck_notification_time     CHECK (sent_at IS NULL OR sent_at >= created_at)
);

CREATE TABLE event_log (
    id           BIGINT       GENERATED ALWAYS AS IDENTITY,
    created_at   TIMESTAMPTZ  NOT NULL DEFAULT now(),
    event_type   event_kind   NOT NULL,
    user_id      INTEGER,
    equipment_id INTEGER,
    incident_id  BIGINT,
    message      VARCHAR(500) NOT NULL,
    details      JSONB,
    CONSTRAINT pk_event_log           PRIMARY KEY (id),
    CONSTRAINT fk_event_log_user      FOREIGN KEY (user_id) REFERENCES users (id)
                                      ON DELETE SET NULL,
    CONSTRAINT fk_event_log_equipment FOREIGN KEY (equipment_id) REFERENCES equipment (id)
                                      ON DELETE SET NULL,
    CONSTRAINT fk_event_log_incident  FOREIGN KEY (incident_id) REFERENCES incident (id)
                                      ON DELETE SET NULL,
    CONSTRAINT ck_event_log_details   CHECK (details IS NULL OR jsonb_typeof(details) = 'object')
);

-- ---------------------------------------------------------------------
-- 7. Індекси (PostgreSQL не створює індекси для зовнішніх ключів автоматично)
-- ---------------------------------------------------------------------
CREATE INDEX ix_users_role             ON users (role_id);
CREATE INDEX ix_equipment_type         ON equipment (type_id);
CREATE INDEX ix_equipment_location     ON equipment (location_id);
CREATE INDEX ix_equipment_status       ON equipment (status) WHERE is_active;
CREATE INDEX ix_threshold_metric       ON threshold (metric_id);
CREATE INDEX ix_reading_read_at_brin   ON metric_reading USING BRIN (read_at);
CREATE INDEX ix_incident_equipment     ON incident (equipment_id, opened_at DESC);
CREATE INDEX ix_incident_threshold     ON incident (threshold_id) WHERE threshold_id IS NOT NULL;
CREATE INDEX ix_incident_ack_user      ON incident (acknowledged_by) WHERE acknowledged_by IS NOT NULL;
CREATE INDEX ix_incident_opened        ON incident (opened_at);
CREATE INDEX ix_incident_new           ON incident (opened_at) WHERE status = 'new';
-- не більше одного відкритого інциденту на пару «обладнання — поріг» (або «обладнання — офлайн»)
CREATE UNIQUE INDEX uq_incident_open   ON incident (equipment_id, incident_type, threshold_id) NULLS
NOT DISTINCT
                                       WHERE status IN ('new', 'escalated', 'acknowledged');
CREATE INDEX ix_notification_antiflood ON notification (incident_id, user_id, channel, created_at
DESC);
CREATE INDEX ix_notification_user      ON notification (user_id);
```

```
CREATE INDEX ix_notification_queued    ON notification (created_at) WHERE status = 'queued';
CREATE INDEX ix_event_log_created      ON event_log (created_at DESC);
CREATE INDEX ix_event_log_type         ON event_log (event_type, created_at DESC);
CREATE INDEX ix_event_log_equipment    ON event_log (equipment_id, created_at DESC) WHERE equipment_id
IS NOT NULL;
CREATE INDEX ix_event_log_user         ON event_log (user_id) WHERE user_id IS NOT NULL;
CREATE INDEX ix_event_log_incident     ON event_log (incident_id) WHERE incident_id IS NOT NULL;

-- ---------------------------------------------------------------------
-- 8. Коментарі до об'єктів (словник даних у самій БД)
-- ---------------------------------------------------------------------
COMMENT ON SCHEMA netguard              IS 'NetGuard Monitor — моніторинг мережевого обладнання та
ДБЖ';
COMMENT ON TABLE  roles                 IS 'Довідник ролей доступу (Сапорт, Адміністратор)';
COMMENT ON TABLE  users                 IS 'Облікові записи користувачів системи';
COMMENT ON TABLE  locations             IS 'Вузли провайдера: ЦОД і районні вузли доступу';
COMMENT ON TABLE  equipment_type        IS 'Довідник типів (моделей) обладнання з профілем SNMP MIB';
COMMENT ON TABLE  metric                IS 'Довідник метрик телеметрії та одиниць виміру';
COMMENT ON TABLE  equipment             IS 'Одиниці мережевого обладнання та ДБЖ, що опитуються за
SNMP';
COMMENT ON TABLE  ups_details           IS 'Додаткові паспортні характеристики ДБЖ (підтип equipment,
1:1)';
COMMENT ON TABLE  threshold             IS 'Порогові значення метрик для типу обладнання';
COMMENT ON TABLE  metric_reading        IS 'Історія показників телеметрії (секціонування за місяцями)';
COMMENT ON TABLE  incident              IS 'Інциденти (аварії) та їх життєвий цикл';
COMMENT ON TABLE  notification          IS 'Сповіщення про інциденти за каналами доставки';
COMMENT ON TABLE  event_log             IS 'Журнал подій: зміни стану, інциденти, дії користувачів';
COMMENT ON COLUMN users.password_hash   IS 'Хеш пароля bcrypt (60 символів); відкритий пароль не
зберігається';
COMMENT ON COLUMN equipment.failed_polls IS 'Кількість поспіль невдалих опитувань; 3 і більше — статус
offline (SR-04.2)';
COMMENT ON COLUMN threshold.min_value   IS 'Нижня межа норми; значення < min_value — порушення';
COMMENT ON COLUMN threshold.max_value   IS 'Верхня межа норми; значення > max_value — порушення';
COMMENT ON COLUMN incident.metric_value IS 'Значення показника, що спричинило інцидент (знімок на
момент відкриття)';
```